Curve Treeを用いた非中央集権型のLinked Dataベース Verifiable Credentialシステム

柳澤 陽希[1,a)]　中西 透[1,b)]　北須賀 輝明[1,c)]　連 卓涛[1,d)]

概要：近年, 非中央集権的な認証として, 第三者機関から発行された証明書をユーザーが保有し, それをサービス提供者に提示することにより属性などの証明を行う Verifiable Credential（VC）という仕組みが注目を集めている. 従来, Linked Data ベースの証明書に対して, 属性の選択的開示を可能にした VC が提案されているが, 署名ベースのため発行者の秘密鍵が必要であり, 発行者は鍵を厳重に長期間管理する必要があるという問題がある. そこで本研究では, Linked Data ベースの証明書に対し Curve Tree によるセットメンバーシップ証明を適用することによって, 選択的開示を可能にしつつ, 長期的な証明書鍵の管理を必要としない非中央集権的な Linked Data ベースの VC システムを提案する. そして, PC 上で実装し, 有効性と実用性の評価を行う.

キーワード：Verifiable Credential, Linked Data, Curve Trees, 非中央集権型匿名属性認証, ブロックチェーン

Decentralized Linked-Data-Based Verifiable Credential System Using Curve Tree

Haruki Yanagisawa[1,a)]　Toru Nakanishi[1,b)]　Teruaki Kitasuka[1,c)]　Zhuotao Lian[1,d)]

Abstract: In recent years, Verifiable Credentials (VC) have attracted attention as a mechanism for decentralized authentications, in which a user holds a credential issued by a third-party issuer and presents it to a service provider to prove attributes and other information. Previously, VC systems based on Linked Data have been proposed, which enables selective disclosure of attributes. However, such signature-based systems require the issuer to strictly hold a private key for a long period of time, which poses a significant problem. In this paper, we therefore propose a decentralized Linked Data-based VC system that applies set membership proofs based on Curve Trees to Linked-Data-based credentials, thereby enabling selective disclosure while eliminating the need for long-term certificate key management. We implemented the proposed system on a PC and evaluate its effectiveness and practicality.

Keywords: Verifiable Credentials, Linked Data, Curve Trees, Decentralized Anonymous Attribute-Based Credential Systems, Blockchain

1. はじめに

オンラインサービスでは, ユーザの属性情報を ID プロバイダが管理し, サービス提供者はユーザのアクセス時に ID プロバイダへ問い合わせをすることによりユーザ認証および属性確認を行う. しかし, この中央集権型のシステムでは ID プロバイダの過失による属性情報の流出や, システム障

[1] 広島大学大学院 先進理工系科学研究科
Graduate School of Advanced Science and Engineering, Hiroshima University
[a)] m261501@hiroshima-u.ac.jp
[b)] t-nakanishi@hiroshima-u.ac.jp
[c)] kitasuka@hiroshima-u.ac.jp
[d)] lian-zhuotao@hiroshima-u.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

害によるサービス停止等の問題がある.

そこで, 非中央集権型の新たなデジタルアイデンティティのアプローチとして, Verifiable Credential（VC）が注目されている. VC では, 発行者から発行された証明書をユーザ自身で管理し, 必要な場面で提示することで自身の属性の証明を行う. 証明書は発行者のディジタル署名が付与されており, それを提示を受けたサービス提供者が検証することにより証明されている内容を確認できる. また, 提示時にはユーザのプライバシー保護の観点から, 提示に不要な属性は秘匿しつつ, 必要な属性のみ開示する選択的開示の機能が必要とされる.

近年 [1] において, Linked Data（LD）というデータ構造（図 1）を持つ証明書に対して選択的開示が可能な VC 方式が提案されている. この方式では, LD を表現する際に, 属性とその関係を RDF トリプルと呼ばれる「主語-述語-目的語」の 3 つ組で表し, その集合を有向グラフ（RDF グラフ）として扱う. さらに, 異なる証明書間において共通のノードを介してグラフを結合し, 証明書間での属性の関係を示すことが可能である.

さらに [2] では, [1] の方式を拡張し, 検証時間がグラフの RDF トリプル数に依存しない LD ベース VC が提案されている. この方式では, セットコミットメントを用いてグラフの頂点, 辺, 辺ラベルの各集合を圧縮した上で発行者が SPS-EQ 署名を付与する. そして, 証明したい属性が圧縮値に含まれていることをゼロ知識証明することにより, グラフサイズに対して $O(1)$ での検証時間を達成している.

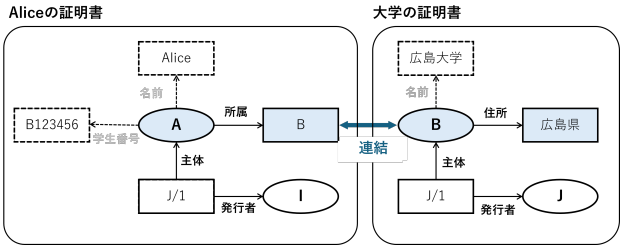


図 1 LD ベース VC での選択的開示の例

しかし, 従来方式 [1], [2] の課題として, 証明書の正当性が発行者の秘密鍵に依存しており, 発行者は長期にわたって鍵を安全に管理する必要がある点が挙げられる. 一方, ブロックチェーン上でコミットメントとして証明書を管理することにより, 長期的な発行者の鍵管理を必要としない非中央集権的な VC システム [3], [4] が提案されている. そこで本研究では, LD ベースの証明書に対して Curve Tree [5] を適用することにより, 発行者による鍵管理が不要な LD ベース VC システムを提案する. Curve Tree は, 集合に特定の要素が含まれることをゼロ知識で効率的に証明できる方式である. Curve Tree では, 集合の要素を楕円曲線上の点として葉ノードに対応付け, マークル木の構築時に木の各層で使用する楕円曲線を交互に切り替えながらノードを圧縮する. そして, 得られた木のルートから特定の葉ノードまでのパスをランダム化しながら証明することによって, セットメンバーシップ証明を行う.

本研究では, さらに提案方式を PC 上で実装し, 証明時間および検証時間を計測して実用性の評価を行う.

2. 数学的準備

2.1 楕円曲線

$\mathbb{F}_q$ における楕円曲線 $\mathbb{E}$ 上の有理点 (X, Y) の集合を $\mathbb{E}[\mathbb{F}_q] \subseteq \mathbb{F}_q \times \mathbb{F}_q$ と定義する. 本稿では, $\mathbb{E}[\mathbb{F}_q]$ 上の点の個数を $p := |\mathbb{E}[\mathbb{F}_q]|$ と表記し, p は素数とする. このとき, $\langle \mathbb{E}[\mathbb{F}_q], + \rangle$ は巡回群となる.

また, $\mathbb{E}[\mathbb{F}_q]$ 上の点 G, スカラー s に対して G のスカラー倍を $[s] \cdot G$ と表す. さらに, スカラーのベクトル $\boldsymbol{s} \in \mathbb{F}_p$, $\mathbb{E}[\mathbb{F}_q]$ 上の点のリスト $\boldsymbol{G}$ に対して, $\boldsymbol{s}$ との内積を $\langle \boldsymbol{s}, \boldsymbol{G} \rangle = \sum_i s_i \cdot G_i$ と表記する.

2.2 安全性仮定

本研究で用いる Curve Tree は, 以下の一般化離散対数仮定に基づき安全性が保証される.

> **一般化離散対数仮定**: $G(1^\lambda)$ を, セキュリティパラメータ λ に対して $(\mathbb{E}, \mathbb{F}_p, \mathbb{F}_q)$ を生成する確率的多項式時間（PPT）アルゴリズムとする. このとき, すべての PPT 攻撃者 $\mathcal{A}$ および整数 $m > 2$ について, 以下の確率は無視できる.

$$\Pr\left[\begin{array}{l|c} \langle \boldsymbol{a}, \boldsymbol{G} \rangle = 0 \in \mathbb{E}[\mathbb{F}_q] & (\mathbb{E}, \mathbb{F}_q, \mathbb{F}_p) \leftarrow \mathcal{G}(1^\lambda) \\ \wedge\ \boldsymbol{a} \neq \boldsymbol{0} \in (\mathbb{F}_p)^m & \boldsymbol{G} \leftarrow \mathbb{E}[\mathbb{F}_q]^m \\ & \boldsymbol{a} \leftarrow \mathcal{A}((\mathbb{E}, \mathbb{F}_q, \mathbb{F}_p), \boldsymbol{G}) \end{array}\right]$$

2.3 コミットメント

コミットメントとは, 送信者がある値を秘匿した状態で受信者に送信（コミット）し, 後にコミット値とコミットメントに使われた乱数が開示された際, 受信者はそれらを用いてコミット値の正当性を検証することができる方式である. コミットメントの安全性を以下に示す.

> **秘匿性（hiding）**：送信者が開示を行うまで, コミットされた値については一切漏れることがない.

> **拘束性（binding）**：コミットメントが送信された後, コミットされた値を変更することはできない.

本研究で提案する方式では, $\mathbb{E}[\mathbb{F}_q]$ 上の Pedersen Commitment を用いる. これは秘密情報をベクトルとして扱い, それらを楕円曲線上の点に集約するものであり, 以下のように定義される.

$$C = \mathsf{Com}(v, r) = \langle \boldsymbol{v}, \boldsymbol{G} \rangle + [r] \cdot H \in \mathbb{E}[\mathbb{F}_q]$$

ここで, $\boldsymbol{v}$ は $\mathbb{F}_p$ の値のベクトル, $\boldsymbol{G}$ は $\mathbb{E}[\mathbb{F}_q]$ 生成元のベクトル, r は $\mathbb{F}_q$ 上の乱数, H は $\mathbb{E}[\mathbb{F}_q]$ の生成元である.

© 2026 Information Processing Society of Japan

2.4 ゼロ知識証明

ゼロ知識証明とは, 秘密の情報がある条件を満たしていることを, その秘密の情報を明かすことなく証明する技術である.

2.4.1 知識の署名（(SPK, signature of knowledge)

証明者がある秘密を知っていることを検証者に証明するための対話型プロトコルとして, 知識の証明（PK, proof of knowledge）がある. これに対し, Fiat-Shamir heuristic によって非対話型に変換し, メッセージ m に対する署名として構成したものを知識の署名（SPK, signature of knowledge), または NIZK arguments と呼ぶ. SPK では, 秘密の証拠情報 w がある関係 $\mathcal{R}(x,w)$ を満たしていることを以下の 2 つのアルゴリズムにより証明する.

- $\mathsf{Prove}(\mathsf{pp},\mathcal{R},x,w)\to\pi$
  公開パラメータ pp, 関係 $\mathcal{R}$, 公開情報 x, 秘密 w を入力として受け取り, 証明 π を出力する.
- $\mathsf{VerProof}(\mathsf{pp},\mathcal{R},x,\pi)\to b\in\{0,1\}$
  公開パラメータ pp, 関係 $\mathcal{R}$, 公開情報 x, 証明 π を入力として受け取り, 検証に成功したら 1 を, そうでなければ 0 を出力する.

本稿では, メッセージ m に対し, 式 $Y=[x]\cdot G$ を満たす秘密 x を知っている者による署名を以下の記法で表現する.

$$SPK\{(x):Y=[x]\cdot G\}(m)$$

2.4.2 Bulletproofs

Curve Tree では, Commit-and-Prove に基づく汎用ゼロ知識証明 (zk-SNARKs) として, Bulletproofs [6] が利用されている. Bulletproofs は R1CS と呼ばれる算術回路で表現された任意の NP 関係を効率的に証明可能であり, セットアップに信頼できる第三者を必要としないという特徴がある. 本稿では, 楕円曲線 $\mathbb{E}$ 上でインスタンス化された Bulletproofs を zk-BP[$\mathbb{E}$] と表記する.

2.5 Curve Tree

Curve Tree は, 信頼できる第三者によるセットアップを必要とすることなく, 集合の中に特定の要素が含まれていることをどの要素かを明かすことなく効率的に証明するシステムである. このシステムでは, 集合サイズ n に対して $O(\log n)$ の証明サイズと sublinear な処理時間を実現している.

2.5.1 楕円曲線の 2-サイクル

2 つの楕円曲線 $(\mathbb{E}_{(\mathrm{evn})},\mathbb{E}_{(\mathrm{odd})})$ と 2 つの素体 $(\mathbb{F}_p,\mathbb{F}_q)$ を考える. このとき, p,q が $p=|\mathbb{E}_{(\mathrm{evn})}[\mathbb{F}_q]|$ かつ $q=|\mathbb{E}_{(\mathrm{odd})}[\mathbb{F}_p]|$ を満たす 2 曲線を構成でき, 2-サイクルと呼ばれる. 2-サイクルでは, 一方の曲線の基礎体は他方の曲線のスカラー数と一致するため, $\mathbb{E}_{(\mathrm{evn})}$ 上の点 (X,Y) の座標は, $\mathbb{E}_{(\mathrm{odd})}$ 上のスカラーとして扱うことができる. $\mathbb{E}_{(\mathrm{evn})}$ と $\mathbb{E}_{(\mathrm{odd})}$ を入れ替えても同様である. 例えば, $[X]\cdot G_1+[Y]\cdot G_2$ （$G_1,G_2\in\mathbb{E}_{(\mathrm{odd})}[\mathbb{F}_p]$ ）は $\mathbb{E}_{(\mathrm{odd})}[\mathbb{F}_p]$ 上の点として計算される.

2.5.2 Curve Tree の定義

以下, 木の特定の層において対応する曲線を $\mathbb{E}_{(_)}$, もう一方の曲線を $\mathbb{E}_{other(_)}$ と表記する. つまり, $\mathbb{E}_{(_)}$ が $\mathbb{E}_{(\mathrm{odd})}$ のとき $\mathbb{E}_{other(_)}$ は $\mathbb{E}_{(\mathrm{evn})}$ となる.

このとき, 木の深さ $D\in\mathbb{N}$, 分岐係数 $l\in\mathbb{N}$, 楕円曲線の 2-サイクル $(\mathbb{E}_{(\mathrm{evn})},\mathbb{E}_{(\mathrm{odd})},\mathbb{F}_p,\mathbb{F}_q)$, および 2 つの楕円曲線上のそれぞれ $2l$ 個の点 $\boldsymbol{G}^{\mathbf{x}}_{(\mathrm{evn})},\boldsymbol{G}^{\mathbf{y}}_{(\mathrm{evn})}\in\mathbb{E}^l_{(\mathrm{evn})}$, $\boldsymbol{G}^{\mathbf{x}}_{(\mathrm{odd})},\boldsymbol{G}^{\mathbf{y}}_{(\mathrm{odd})}\in\mathbb{E}^l_{(\mathrm{odd})}$ に対し, Curve Tree は以下のように定義される.

葉ノード: $(0,l,\mathbb{E}_{(_)},\mathbb{E}_{other(_)})$-Curve Tree
葉ノードはすべて楕円曲線上の点 $C\in\mathbb{E}_{(_)}$ が対応する.
内部ノード: $(D,l,\mathbb{E}_{(_)},\mathbb{E}_{other(_)})$-Curve Tree
内部ノード C は l 個の $(D-1,l,\mathbb{E}_{other(_)},\mathbb{E}_{(_)})$-Curve Tree のルートノード $(X_1,Y_1),\cdots,(X_l,Y_l)$ に対して, 生成元 $\boldsymbol{G}^x_{(_)},\boldsymbol{G}^y_{(_)}\in\mathbb{E}_{(_)}$ と座標ベクトル $\boldsymbol{X}=(X_1,\cdots,X_l),\boldsymbol{Y}=(Y_1,\cdots,Y_l)\in\mathbb{F}_q^l$ の内積による Pedersen Commitment を計算することにより, 以下のように定義される.

$$C=\langle\boldsymbol{X},\boldsymbol{G}^x_{(_)}\rangle+\langle\boldsymbol{Y},\boldsymbol{G}^y_{(_)}\rangle$$

また, ある葉ノードに対するメンバーシップ証明において, 木のルートからその葉ノードまでのパスの各層でノード同士の親子関係は以下のような $\mathcal{R}^{(\text{single-level, }(\mathrm{evn}))}$（もしくは $\mathcal{R}^{(\text{single-level, }(\mathrm{odd}))}$）で定義される. ここで, C および $\hat{C}$ は親ノードおよび子ノードのコミットメントを再ランダム化したコミットメントである.

$$\mathcal{R}^{(\text{single-level},\mathbb{E}_{(_)})}:=\left\{\begin{array}{l}\begin{pmatrix}i,r,\delta\\ \boldsymbol{X},\boldsymbol{Y}\end{pmatrix}:\ \begin{array}{l}C=\langle\boldsymbol{X},\boldsymbol{G}^x_{(_)}\rangle\\ \quad+\langle\boldsymbol{Y},\boldsymbol{G}^y_{(_)}\rangle\\ \quad+[r]\cdot H_{(_)}\in\mathbb{E}_{(_)}\end{array}\\ \\ \hat{C}=(\boldsymbol{X}_i,\boldsymbol{Y}_i)+[\delta]\cdot H_{\mathrm{other}(_)}\in\mathbb{E}_{\mathrm{other}(_)}\end{array}\right\}$$

そして, 証明したい葉までのパス上のすべての親子関係を偶数層, 奇数層ごとにそれぞれ
$\mathcal{R}^{(\text{evn-levels})}:=\left\{\wedge_{j\in\{0,2,\ldots,D-2\}}\mathcal{R}^{(\text{single-level},\mathbb{E}_{(\mathrm{evn})})}\right\}$,
$\mathcal{R}^{(\text{odd-levels})}:=\left\{\wedge_{j\in\{1,3,\ldots,D-1\}}R^{(\text{single-level},\mathbb{E}_{(\mathrm{odd})})}\right\}$
の 2 つにまとめ, これらを Bulletproofs により証明する.

2.5.3 Curve Tree の証明システムのアルゴリズム

Curve Tree に対する証明システムは以下のアルゴリズムから構成される.

- $\mathsf{SelRerand.Setup}(1^\lambda)\to\mathsf{pp}$
  セキュリティパラメータ λ に対し, $\boldsymbol{G}^{(\mathrm{evn})}\in$

© 2026 Information Processing Society of Japan

$\mathbb{E}_{(\mathrm{evn})}^{N_{urs}}, \boldsymbol{G}^{(\mathrm{odd})} \in \mathbb{E}_{(\mathrm{odd})}^{N_{urs}}, \boldsymbol{H}^{(\mathrm{evn})} \in \mathbb{E}_{(\mathrm{evn})}, \boldsymbol{H}^{(\mathrm{odd})} \in \mathbb{E}_{(\mathrm{odd})}$ を生成し, それらをまとめて公開パラメータ pp として返す. ここで, $N_{urs} \in \mathbb{N}$ は圧縮関数での生成元の個数である.

- SelRerand.Comm(pp, $v_{\mathrm{leaf}} \in \mathbb{F}_{|\mathbb{E}_{(\mathrm{evn})}|}, o \in \mathbb{F}_{|\mathbb{E}_{(\mathrm{evn})}|}$) $\to C$
  葉 v_{leaf} を入力として, 乱数 o を使ってコミットメント C を計算する.
- SelRerand.Rerand(pp, $C \in \mathbb{E}_{(\mathrm{evn})}, r \in \mathbb{F}_{|\mathbb{E}_{(\mathrm{evn})}|}$) $\to \hat{C}$
  コミットメント C に対し, 乱数 r を使って再ランダム化する. $\mathbb{E}_{(\mathrm{odd})}$ に対しても同様に定義される.
- SelRerand.Accum(pp, $S' = \{C_1, \ldots, C_n\}$) $\to rt$
  コミットメント集合 S' を入力として, 2.5.2 に従って Curve Tree の木を構成し, 木のルート rt を出力する.
- SelRerand.Prove(pp, $S, C_{\mathrm{leaf}}, r^{(D)}$) $\to \pi^*$
  ルート rt から証明対象の葉 C_{leaf} までのパスを特定する. パス上のノードをルートから順に $C^{(0)}, \ldots, C^{(D)}$ とする（ここで $C^{(0)} = rt, C^{(D)} = C_{\mathrm{leaf}}$ である）. 証明者は, 各層のノード $C^{(d)}$ を乱数 $r^{(d)}$ で再ランダム化した値 $\hat{C}^{(d)}$ を計算する. そして, $\hat{C}^{(d)}$ と $\hat{C}^{(d-1)}$ の親子関係を示す式 $\mathcal{R}^{(\text{evn-levels})}, \mathcal{R}^{(\text{odd-levels})}$ に対し, 偶数層・奇数層それぞれの曲線上で Bulletproofs を用いてゼロ知識証明 $\pi_{\mathrm{evn}}, \pi_{\mathrm{odd}}$ を作成し, 証明 $\pi^* = (\hat{C}^{(0)}, \cdots, \hat{C}^{(D-1)}, \pi_{\mathrm{evn}}, \pi_{\mathrm{odd}})$ を出力する.
- SelRerand.Vfy(pp, $rt, \hat{C}_{\mathrm{leaf}}, \pi^*$) $\to \{0, 1\}$
  検証者は, 入力されたルートノード rt, 証明対象の葉ノードの再ランダム化 $\hat{C}_{\mathrm{leaf}}$, 証明 π^* において, 偶数層および奇数層の Bulletproofs の証明 $\pi_{\mathrm{evn}}, \pi_{\mathrm{odd}}$ に対して検証を実行する. 両方の検証が受理された場合のみ 1 を, そうでなければ 0 を出力する.

2.5.4 Curve Tree の安全性

Curve Tree の安全性は以下のように定義される.

- 束縛性（binding）
  任意の PPT 攻撃者 $\mathcal{A}$ が, 集合に含まれないコミットメント $\hat{C}$ の所属を受理させる証明を生成できる確率は, 無視できるほど小さい.
- ゼロ知識性（zero-knowledge）
  任意の PPT 攻撃者 $\mathcal{A}$ に対して, シミュレータ $\mathcal{S}$ によって C_{leaf}, $r^{(D)}$ を使用せずに生成された証明と SelRand.Prove により生成された証明は識別不可能である.

3. 先行研究

3.1 従来方式の概要

選択的開示が可能な LD ベース VC システムとして, ペアリングベースセットコミットメントを用いた三宅らの方式 [2] が提案されている. この方式では Linked Data は RDF トリプルから構成される RDF グラフにより表現される. この形式では, 一つの属性を（主語, 述語, 目的語）という 3 つ組で表し, 主語, 述語, 目的語のそれぞれを RDF 項とよぶ. このとき, 主語を始点, 目的語を終点とし, 述語を辺ラベルとすると RDF トリプルをラベル付き有向辺とみなすことができる. そして, RDF トリプルの集合を RDF グラフと呼び, これは有向グラフとみなせる. 従来方式では, この RDF グラフに対してセットコミットメントにより圧縮した上で発行者が署名することによりクレデンシャルを生成し, その署名とセットコミットメントのゼロ知識証明を用いて選択的開示を実現している.

3.2 従来方式の課題

従来方式の課題として, 認証の正当性が各発行者の秘密鍵の安全性に依存している点が挙げられる. 従来方式ではクレデンシャルを発行する際に発行者の秘密鍵を用いて署名を行うが, このとき発行者の秘密鍵が漏洩した場合, 不正なクレデンシャルが発行されてしまうリスクがある. したがって, 安全性を維持するためには発行者が秘密鍵を長期にわたって厳重に管理する必要があり, これには多大なコストを要するという課題がある.

4. 非中央集権型 LD ベース VC システムのモデル

4.1 アルゴリズムの定義

本稿で提案する非中央集権型 Linked Data ベース Verifiable Credential システムは以下のアルゴリズムから構成される.

このモデルは, [3] の非中央集権型匿名属性認証システムのモデルを LD ベース VC を扱えるように拡張している.

- Setup(1^λ) $\to params$
  セキュリティパラメータ 1^λ に対して公開パラメータ $params$ を生成する.
- KeyGen($params$) $\to sk$
  $params$ に対しユーザの秘密鍵 $sk \in \mathbb{Z}_p$ を生成する.
- MintCred($params, sk, \boldsymbol{V}, aux$) $\to (c, \pi_M)$
  $params$, 秘密鍵 sk, RDF グラフ $\boldsymbol{V} = \{v_1, v_2, \cdots, v_n\}$, およびその正しさを保証する補助情報 aux に対して, クレデンシャル c を計算する. そして, クレデンシャル c と、それが正当であることを示す証明 π_M を出力する.
- MintVerify($params, \boldsymbol{V}, c, aux, \pi_M$) $\to \{0, 1\}$
  証明 π_M を $params, \boldsymbol{V}, c, aux$ を用いて検証し, 検証に成功するなら 1 を出力する. 検証ノードはクレデンシャル c をブロックチェーンの台帳に載せる.

© 2026 Information Processing Society of Japan

- Show$(params, \boldsymbol{C}, c, sk, \boldsymbol{V}, \boldsymbol{open}) \rightarrow \pi_S$
台帳に登録されているクレデンシャルの集合を $\boldsymbol{C}$, $\boldsymbol{V}$ のうち開示 RDF 項のインデックス集合 $\boldsymbol{open}_i$ を要素にもつベクトル $\boldsymbol{open} = (\boldsymbol{open}_1, \cdots, \boldsymbol{open}_n)$ を入力とする. たとえば, $\boldsymbol{v}_1 = (s_1, p_1, o_1)$ の p_1, o_1 のみを開示する場合, $\boldsymbol{open}_1 = \{1, 2\}$ となり, $\boldsymbol{v}_2$ のすべての成分を秘匿する場合は $open_2 = \emptyset$ となる.

  自分のクレデンシャル c が $\boldsymbol{C}$ に含まれることと, c が秘密情報 $sk, \boldsymbol{open}$ で開示される RDF 項から構成されることを示す証明 π_S を出力する.

- ShowVerify$(params, \boldsymbol{open}, \{v_{i,j}\}_{j \in \boldsymbol{open}_i}, \pi_S, \boldsymbol{C}) \rightarrow \{0, 1\}$
c と $\boldsymbol{open}$ および開示 RDF 項 $\{v_{i,j}\}_{j \in \boldsymbol{open}_i}$ を用いて証明 π_S を検証する. 検証が成功するなら 1 を, そうでなければ 0 を出力する.

4.2 システムモデルとプロトコル

本システムは, ユーザーがクレデンシャルをブロックチェーンの台帳へ登録するプロトコルと, それを参照してサービス提供者にクレデンシャルを提示し, 認証を行うプロトコルの二つから構成される. 事前に Setup により $params$ が生成されているものとする.

登録プロトコル: ユーザは KeyGen, MintCred を実行し, 自身のクレデンシャル c と, その正当性を示す証明 π_M を生成した上でブロックチェーン上へ送付し, 登録要求を行う. ブロックチェーンの各ノードは MintVerify を実行して π_M を検証し, 検証が成功した場合は登録を受理し, そうでなければ登録を拒否する.

認証プロトコル: ユーザは Show を実行し,「クレデンシャルがブロックチェーン上に登録されていること」と「開示 RDF 項を含むクレデンシャルを構成する秘密鍵と秘匿する RDF 項を知っていること」の二つを示す証明を作成し、サービス提供者に送信する. サービス提供者は ShowVerify を実行し, チェーン上のクレデンシャル集合をもとに, ユーザから送られてきた証明を検証する. 検証が成功した場合は認証を成功とし, そうでなければ失敗とする.

4.3 安全性定義

偽造不可能性: 正当な秘密鍵 sk を持たない, あるいは開示される RDF 項を含むクレデンシャルが c に含まれない場合, その攻撃者が ShowVerify を通過する証明 π_S を生成することは, 計算量的に困難である.

匿名性: Show プロトコルにおいて正当な証明 π_S が与えられたとき, 検証者は開示された RDF 項 以外の情報（ユーザの秘密鍵 sk, sk_c, 秘匿 RDF 項, および台帳 $\boldsymbol{C}$ 内のどのクレデンシャルを使用したか）を知ることはできない.

5. 提案方式

5.1 提案方式の概要

本稿では, 非中央集権型匿名属性認証 [4] において全ユーザのクレデンシャル集合に対して用いられている Curve Tree を, 各クレデンシャル内の RDF トリプルにも適用する. これにより RDF トリプルのコミットメント集合を 1 つのルートへ圧縮し, 信頼できる第三者を必要としない選択的開示可能な LD ベース VC を実現する. クレデンシャルの登録時には, ユーザは各 RDF トリプルをベクトルとして扱い, それらに対する Pedersen Commitment を葉ノードとする Curve Tree を構築し, そのルートコミットメントの正当性を検証した上でブロックチェーンへ登録する. 提示時には, 全ユーザのクレデンシャルのコミットメントから構成した Curve Tree の木に対し Select-and-Rerandomize を適用して自身のクレデンシャルが台帳に含まれることを証明する. さらに, そのクレデンシャルをルートとする木に対し, 開示するトリプルに対してのみ再度 Select-and-Rerandamize を行うことで, 選択的開示を実現する.

5.2 提案方式のアルゴリズム

以下に提案方式のアルゴリズムを示す.

- Setup$(1^\lambda) \rightarrow params$

  セキュリティパラメータ λ を入力として, Curve Tree の公開パラメータ

$$\begin{aligned} params = (&\boldsymbol{G}_{\mathrm{evn}} = (G_0^{\mathrm{evn}}, \cdots, G_l^{\mathrm{evn}}), \\ &\boldsymbol{G}_{\mathrm{odd}} = (G_0^{\mathrm{odd}}, \cdots, G_l^{\mathrm{odd}}), \\ &H_{\mathrm{evn}}, H_{\mathrm{odd}}, p, q) \end{aligned}$$

  を出力する. ここで, l は Curve Tree における分岐係数の最大値である.

- KeyGen$(params) \rightarrow sk$
ユーザは自身の秘密鍵 $sk \in \mathbb{Z}_p$ をランダムに選ぶ.

- MintCred$(params, sk, \boldsymbol{V}, aux) \rightarrow (c, \pi_M)$
$\boldsymbol{V} = (\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n)$ を, 証明する RDF グラフにおける RDF トリプルのベクトル $\boldsymbol{v}_i = (s_i, p_i, o_i)$ のリストとする.

(1) インデックス i に対し, 以下の処理を行う.

  (i) sk と各 RDF トリプル $\boldsymbol{v}_i$ の Pedersen コミットメント（ただし, 乱数は 0）を以下のように計算する.

$$\begin{aligned} C_i &= \langle (s_i, p_i, o_i, sk), (G_0^{\mathrm{evn}}, \cdots, G_3^{\mathrm{evn}}) \rangle \\ &= s_i \cdot G_0^{\mathrm{evn}} + p_i \cdot G_1^{\mathrm{evn}} + o_i \cdot G_2^{\mathrm{evn}} + sk \cdot G_3^{\mathrm{evn}} \end{aligned}$$

© 2026 Information Processing Society of Japan

(ii) C_i が正しい sk を知るユーザによってつくられたことを示す証明 $\pi_{\mathrm{acc},i}$ を以下のように作成する. ここで, aux は補助情報である.

$$\pi_{\mathrm{acc},i} := SPK\{(sk) : C_i = \langle (s_i, p_i, o_i, sk), (G_0^{\mathrm{evn}}, \cdots, G_3^{\mathrm{evn}}) \rangle\}(aux)$$

(2) コミットメント集合 $\{C_1, C_2, \cdots C_n\}$ に対して Curve Tree の Accum を適用して木を生成し, 得られたルートを c とする.

(3) c と $\pi_M = \{(C_1, \pi_{\mathrm{acc},1}), \cdots, (C_n, \pi_{\mathrm{acc},n})\}$ を出力する.

- MintVerify$(params, \boldsymbol{V}, c, aux, \pi_M) \to \{0, 1\}$

(1) コミットメント集合 $\{C_1, C_2, \cdots C_n\}$ に対して Curve Tree の Accum を適用して木を生成し, そのルートが c と等しいか検証する.

(2) SPK $\pi_{\mathrm{acc},1}, \cdots, \pi_{\mathrm{acc},n}$ の正当性を検証する. (1) に加えて $\pi_{\mathrm{acc},1}, \cdots, \pi_{\mathrm{acc},n}$ の検証に成功したら 1 を, 失敗なら 0 を出力して終了する.

このアルゴリズムが 1 を出力をした場合, ブロックチェーンの検証者ノードは c をユーザのクレデンシャルとしてブロックチェーンの台帳へ登録する.

- Show$(params, \boldsymbol{C}, c, sk, \boldsymbol{V}, \boldsymbol{open}) \to \pi_S$

(1) (i) ブロックチェーン上に登録されているクレデンシャルの集合 $\boldsymbol{C}$ を入力として Accum に渡して Curve Tree の木を生成し, 得られたルートを srt とする.
自身のクレデンシャル c に対し, 根 srt から c までのパスを $c_0, c_1, \cdots, c_{D-1}, c_D$ とする. ここで, srt がパス上の c_0, c が c_D にそれぞれ対応する.

(ii) $i = 2, 4, 6, \cdots, j = 1, 3, 5 \cdots$ とする.
このとき, $r_0 = 0, r_i \in \mathbb{Z}_p, r_j \in \mathbb{Z}_q$ をランダムに選び, 以下の式を用いて $c_1, \cdots, c_D$ の再ランダム化 $c_0^*, c_1^*, \cdots, c_D^*$ を計算する.

$$c_i^* = c_i + [r_i] \cdot H_i^{\mathrm{evn}}$$
$$c_j^* = c_j + [r_j] \cdot H_j^{\mathrm{odd}}$$

(iii) c_i^*, c_j^* がそれぞれ c_i, c_j からつくられたコミットメントであることと, c_i^* が c_{i+1}^* の親ノードであり, c_j^* が c_{j+1}^* の親ノードであることを示す証明 π_c を, Prove を用いて以下のように生成する.

$$\pi_{\mathrm{evn}} \leftarrow \mathsf{zk\text{-}BP}[\mathbb{E}_{\mathrm{evn}}].\mathsf{Prove}(params, \mathcal{R}^{(\text{evn-levels})}, \mathsf{x}_{\mathrm{evn}}, \mathsf{w}_{\mathrm{evn}})$$
$$\pi_{\mathrm{odd}} \leftarrow \mathsf{zk\text{-}BP}[\mathbb{E}_{\mathrm{odd}}].\mathsf{Prove}(params, \mathcal{R}^{(\text{odd-levels})}, \mathsf{x}_{\mathrm{odd}}, \mathsf{w}_{\mathrm{odd}})$$

そして, π_{rt} を以下のように設定する.

$$\pi_{rt} = (c_0^*, \cdots, c_D^*, \pi_{\mathrm{evn}}, \pi_{\mathrm{odd}})$$

ここで, $c_D^* = c_D + [r_D] \cdot H_D^{\mathrm{odd}}$ であるが, $c_D = c$ より, c_D^* は c を再ランダム化したものである.

(2) c_D^* を rt' とおく. $\boldsymbol{open}$ を参照し, $\boldsymbol{open}_i \neq \emptyset$ なるインデックス i に対し以下の処理を行い, rt' から各 C_i までのパスを証明する.

(i) 開示する成分をもつベクトルのコミットメント C_i までのパスを $c_0', c_1', \cdots, c_{D-1}', c_D'$ とおく. ここで, c_0' が rt' に, c_D' が C_i に対応する.

(ii) $k = 2, 4, 6, \cdots, l = 1, 3, 5 \cdots$ とする.
このとき, $r_0 = 0, r_k \in \mathbb{Z}_p, r_l \in \mathbb{Z}_q$ をそれぞれ選び, 以下の式を用いて $c_1', \cdots, c_{D-1}', c_D'$ の再ランダム化 ${c_0'}^*, {c_1'}^*, \cdots, {c_{D-1}'}^*, {c_D'}^*$ を計算する.

$${c_k'}^* = c_k' + [r_k] \cdot H_k^{\mathrm{evn}}$$
$${c_l'}^* = c_l' + [r_l] \cdot H_l^{\mathrm{odd}}$$

(iii) ${c_k'}^*, {c_l'}^*$ がそれぞれ c_k', c_l' を再ランダム化したコミットメントであることと, ${c_k'}^*$ が ${c_{k+1}'}^*$ の親ノードであり, ${c_l'}^*$ が ${c_{l+1}'}^*$ の親ノードであることを示す証明を, Prove を用いて以下のように生成する.

$$\pi_{\mathrm{evn},i} \leftarrow \mathsf{zk\text{-}BP}[\mathbb{E}_{\mathrm{evn}}].\mathsf{Prove}(params, \mathcal{R}^{(\text{evn-levels})}, \mathsf{x}'_{\mathrm{evn}}, \mathsf{w}_{\mathrm{evn}})$$
$$\pi_{\mathrm{odd},i} \leftarrow \mathsf{zk\text{-}BP}[\mathbb{E}_{\mathrm{odd}}].\mathsf{Prove}(params, \mathcal{R}^{(\text{odd-levels})}, \mathsf{x}'_{\mathrm{odd}}, \mathsf{w}_{\mathrm{odd}})$$

そして, 以下のようにまとめる.

$$\pi_{c,i} = ({c_0'}^*, \cdots, {c_D'}^*, \pi_{\mathrm{odd},i}, \pi_{\mathrm{evn},i})$$

(iv) $\boldsymbol{v}_i = (s_i, p_i, o_i)$ の j 番目の要素を $v_{i,j}$ とする. ${c_D'}^*$ が $\boldsymbol{v}_i$ のコミットメント C_i を再ランダム化したものであり, 開示する RDF 項 $v_{i,j}$ $(j \in \boldsymbol{open}_i)$ が含まれていることを示す証明 $\pi_{\mathrm{sel},i}$ を以下のように構成する.

$$\begin{aligned}\pi_{\mathrm{sel},i} := SPK\Big\{ & (sk, r_D, \{v_{i,j}\}_{j \notin \boldsymbol{open}_i}) \\ & : {c_D'}^* - \Big(\textstyle\sum_{j \in \boldsymbol{open}_i} [v_{i,j}] \cdot G_j^{\mathrm{evn}}\Big) \\ & = \Big(\textstyle\sum_{j \notin \boldsymbol{open}_i} [v_{i,j}] \cdot G_j^{\mathrm{evn}}\Big) \\ & + [sk] \cdot G_3^{\mathrm{evn}} + [r_D] \cdot H_D \Big\}\end{aligned}$$

(3) 証明 $\pi_i := (\pi_{c,i}, \pi_{\mathrm{sel},i})$ とまとめる.

(4) $\pi_S := (\pi_{rt}, \{\pi_i\}_{i | \boldsymbol{open}_i \neq \emptyset})$ として出力する.

- ShowVerify$(params, \boldsymbol{open}, \{v_{i,j}\}_{j \in \boldsymbol{open}_i}, \pi_S, \boldsymbol{C})$

© 2026 Information Processing Society of Japan

$\to \{0,1\}$

(1) クレデンシャル集合 $\boldsymbol{C}$ に対して Accum を適用し, Curve Tree の木を構成する. 得られたルートを vrt とする.

(2) π_{rt} を検証する. π_{rt} 中の $c_1^*, \cdots, c_D^*$ に加えて $c_0^* = vrt$ をセットし, 以下のように偶数層と奇数層に分けて証明 $\pi_{\text{evn}}, \pi_{\text{odd}}$ を検証する.

$$b_{\text{evn},rt} \leftarrow \mathsf{zk\text{-}BP}[\mathbb{E}_{\text{evn}}].\mathsf{VerProof}(params, \mathcal{R}^{(\text{evn-levels})}, \mathsf{x}_{\text{evn}}, \pi_{\text{evn}})$$

$$b_{\text{odd},rt} \leftarrow \mathsf{zk\text{-}BP}[\mathbb{E}_{\text{odd}}].\mathsf{VerProof}(params, \mathcal{R}^{(\text{odd-levels})}, \mathsf{x}_{\text{odd}}, \pi_{\text{odd}})$$

結果を $b_{rt} = b_{\text{evn},rt} \wedge b_{\text{odd},rt}$ とし, $b_{rt} \neq 1$ ならば 0 を出力して終了する.

(3) c_D^* に対し, $vrt' = c_D^*$ とセットする. $\boldsymbol{open}$ を参照し, $\boldsymbol{open}_i \neq \emptyset$ なるインデックス i に対し, 以下の処理を適用する.

(i) $\pi_{c,i}$ を検証する. 以下のように偶数層と奇数層に分けて証明 $\pi_{\text{evn},i}, \pi_{\text{odd},i}$ を検証する.

$$b_{\text{evn},i} \leftarrow \mathsf{zk\text{-}BP}[\mathbb{E}_{\text{evn}}].\mathsf{VerProof}(params, \mathcal{R}^{(\text{evn-levels})}, \mathsf{x}'_{\text{evn}}, \pi_{\text{evn},i})$$

$$b_{\text{odd},i} \leftarrow \mathsf{zk\text{-}BP}[\mathbb{E}_{\text{odd}}].\mathsf{VerProof}(params, \mathcal{R}^{(\text{odd-levels})}, \mathsf{x}'_{\text{odd}}, \pi_{\text{odd},i})$$

結果を $b_{c,i} = b_{\text{evn},i} \wedge b_{\text{odd},i}$ とする.

(ii) SPK $\pi_{\text{sel},i}$ を検証し, 結果を $b_{rand,i}$ へ代入する.

(iii) $b_{c,i} \wedge b_{\text{sel},i} \neq 1$ ならば, 0 を出力する.

(4) すべての i に対し (3) が成功したら, 検証に成功したとみなし 1 を出力して終了する.

5.3 提案方式の安全性

提案方式について, 以下の二つが成り立つ.

- **偽造不可能性**

  一般化離散対数仮定のもとで Curve Trees は束縛性を満たすことより, クレデンシャル集合 $\boldsymbol{C}$ に含まれていないクレデンシャル c から, 受理される証明 π_{rt} を生成できない. また, その c に含まれていないコミットメント C_i から受理される証明 $\pi_{c,i}$ も生成できない. さらに, SPK の健全性から, C_i の再ランダム化 ${c'_D}^*$ に対して, C_i でコミットされていない RDF 項 $\hat{v}_{i.j}$ を開示する正当な SPK$\pi_{\text{sel},i}$ も生成できない. このことから, 偽造不可能性が成り立つ.

- **匿名性**

  証明 π_{rt}, $\pi_{c,i}$ において, Curve Trees がゼロ知識性をもつことにより, 検証者は追加の情報を知ることができない. 証明 $\pi_{\text{sel},i}$ においても, SPK がゼロ知識性を満たすため同様である. 以上より, 匿名性が成り立つ.

6. 実装と評価

6.1 実装方法

本研究では, GitHub で公開されている CurveTree のソースコード† を拡張することで提案方式を実装した. 実装環境については以下の表 1 の通りである.

表 1 実装環境

OS	Windows11 Home（25H2）
CPU	AMD Ryzen7 5700 8-Core Processor
メモリ	32.00 GB
プログラミング言語	Rust
楕円曲線	Pasta Cycle

測定では証明書内に含まれる RDF トリプルの総数（トリプル数）, 少なくとも一つの RDF 項を開示する RDF トリプル数（開示数）, クレデンシャルがブロックチェーン上に登録されているユーザーの数（ユーザー数）の 3 つのパラメータを変化させて処理時間の比較を行った. 従来方式 [2] についても表 1 の環境で測定している.

6.2 実験結果

6.2.1 RDF トリプルの総数に対する処理時間の比較

図 2, 3 は, 開示数 5, ユーザー数 1024 で固定し, トリプル数を変化させた際の処理時間を従来方式 [2] と提案方式で比較したものである.

図 2 より, 従来方式ではトリプル数に比例して証明時間が線形に増加しているのに対し, 提案方式では, トリプル数 1200 程度までの範囲内では 4.5s 以下に保たれている. これは, 証明・検証コストが Bulletproofs [6] で証明する制約数に依存するためである. Bulletproofs では, 制約数に対して再帰的に 2 で割りながら証明を行っていく関係上, 制約数が 2^{n-1} から 2^n の範囲では処理時間が一定で, 2^n を超えると増加する. そのため, トリプル数 128 で一度処理時間が増えている. 一方, Curve Tree においては制約数が $D(912 + l - 1)$ に応じて変化するため, 適切に D, l を設定することで, 制約数が 2 のべき乗を超えないよう抑えることができる. 本測定では, トリプル数 N に対して, $N = 512$ を境に処理時間の大小関係が逆転するため, $N < 512$ のとき, $D = 1, l = N$. $N \geq 512$ のとき, $D = 2, l = 2^{\lceil \log_2 \sqrt{N} \rceil}$ を設定している. これにより, $N < 1200$ の範囲では約 4.5s 以下の処理時間を達成しており, トリプル数 256 までは従来方式の方が高速なものの, それ以上においては提案方式が高速である.

検証時間については従来方式は一定時間を実現しているのに対し, 提案方式では証明時間と同様の傾向となってい

† https://github.com/simonkamp/curve-trees

© 2026 Information Processing Society of Japan

る．検証時間の実測値としては従来方式の方が高速であるが，提案方式でも 60ms 程度であり十分高速である．

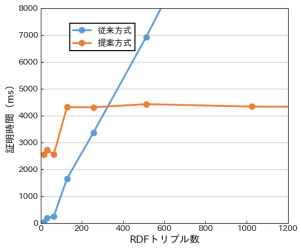


図 2　証明時間の比較

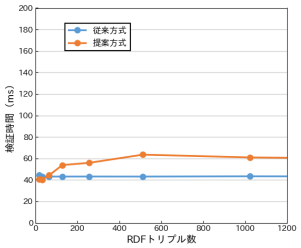


図 3　検証時間の比較

6.2.2　開示する RDF トリプル数に対する処理時間の比較

図 4, 5 は，トリプル数を 256，ユーザー数を 1024 に固定し，開示数を変化させて処理時間を比較した結果である．

両方式ともに証明時間は開示数に対して線形に増加していることがわかる．これは，従来方式では開示する RDF トリプル・項ごとにセットコミットメントの検証をするためであり，提案方式では開示対象の各 RDF トリプルに対して個別に Select-and-Rerandomize によるパス証明と SPK の生成を行う必要があるためである．提案方式の方が処理時間が大きく，この改善は今後の課題である．

検証時間についても同様の理由で線形に増加するが，こちらは 100ms 程度に抑えられており，十分実用的であると考えられる．

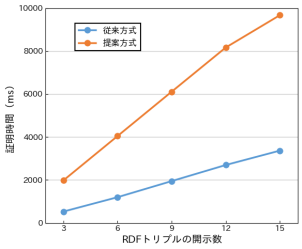


図 4　証明時間の比較

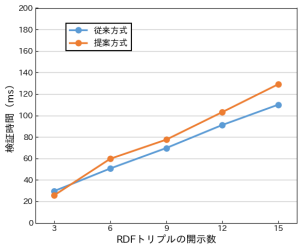


図 5　検証時間の比較

6.2.3　ユーザー数に対する処理時間の比較

図 6 は，トリプル数を 256，開示数を 5 に固定し，ユーザー数を変化させた際の処理時間の推移を，提案方式において上層の木の生成処理を含む場合と含まない場合において比較したものである．なお，本測定においてはユーザー数 M に対して，$D = 2, l = 2^{\lceil \log_2 \sqrt{M} \rceil}$ を設定している．証明時間において，木の生成を含まない場合は sublinear な処理時間を達成しているが，生成を含む場合は M が大きくなると線形増加に変化する．これは，木の生成処理に $O(M)$ の時間がかかり，$M > 50000$ では木の生成コストが支配的になるためである．検証時間も同様の理由で，生成を含めると線形に処理時間が増加する．

この結果から，実運用においては，登録された属性やユーザーに変化がない場合には一度生成された木を繰り返し再利用することで，ユーザー数の規模に関わらず高速に検証を行えると考えられる．

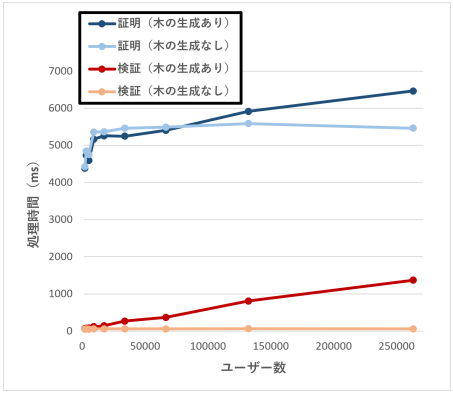


図 6　ユーザー数に対する証明時間・検証時間

7.　まとめ

本研究では，Curve Tree を適用することにより，発行者の署名に依存せず証明書の正当性を検証できる非中央集権型の LD ベース VC システムを提案した．処理時間については，トリプル数が実用上で想定される範囲内であれば証明時間・検証時間ともに総数に依存しない．また，一度生成した木を再利用すれば，ユーザー数による影響も抑えられることが確認された．一方，開示数には大きく影響を受けてしまうため，この部分は今後の課題である．

参考文献

[1] D. Yamamoto, Y. Suga, and K. Sako. ”Formalising linked-data based verifiable credentials for selective disclosure, ” IEEE European Symposium on Security and Privacy Workshops EuroS(&PW), pp. 52–65, 2022.

[2] R. Miyake, T. Nakanishi, T. Kitasuka and Z. Lian, ”Linked-Data based Verifiable Credentials for Selective Disclosure with Efficient Verification,” 2025 Thirteenth International Symposium on Computing and Networking (CANDAR2025), pp. 153-159, 2025.

[3] C. Garman, M. Green, and I. Miers, “Decentralized anonymous credentials, ” NDSS2014, 2014

[4] 渡部 彰馬，中西 透，北須賀 輝明，連 卓涛，”Curve Tree を用いた非中央集権型匿名属性認証における検証時間の削減,” コンピュータセキュリティシンポジウム 2025(CSS2025), pp.1080-1087, 2025.

[5] M. Campanelli, M. Hall-Andersen, and S. H. Kamp, “Curve trees: Practical and transparent zero-knowledge accumulators,” USENIX Security 2023, pp. 4391–4408, 2023.

[6] B. Büsnz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille and G. Maxwell , ”Bulletproofs: Short Proofs for Confidential Transactions and More,” 2018 IEEE Symposium on Security and Privacy (SP2018), pp. 319–338, 2018.

© 2026 Information Processing Society of Japan